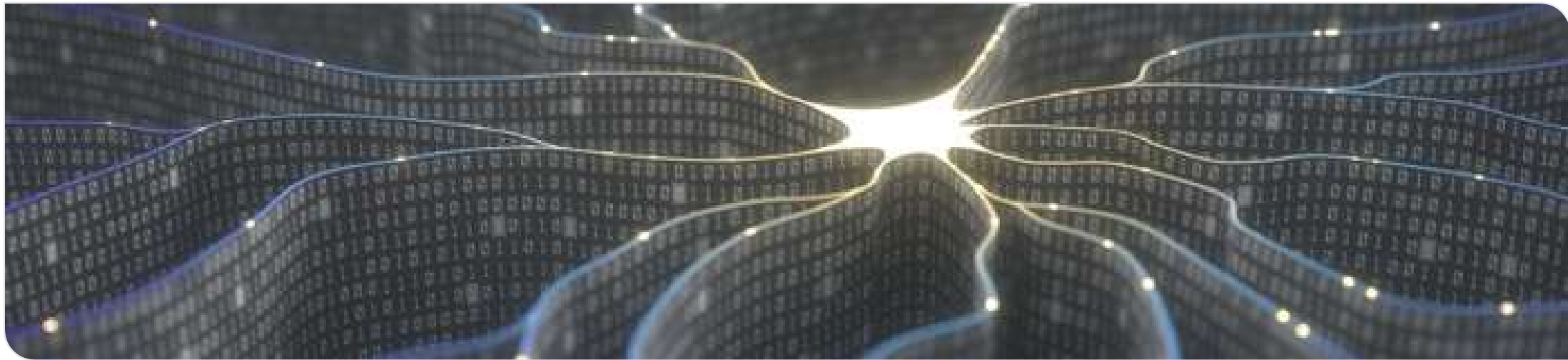


Science Camp KI 2026



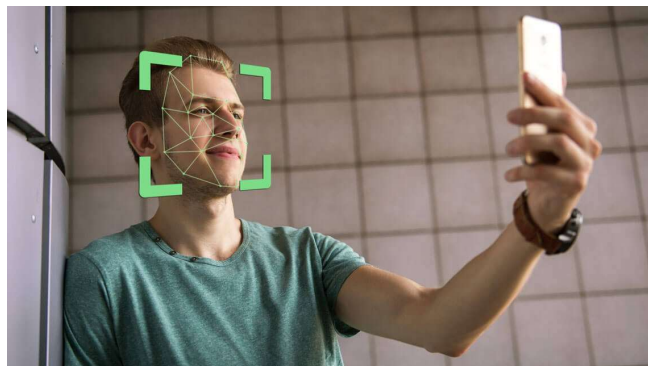
Tagesplan

- **09:00 – 12:00:** Theoretische Einführung
- **12:00 – 13:30:** Mittagspause
- **13:30 – 14:30:** Abschluss Theorieteil
- **14:30 – 16:00:** Beginn Projekt: Ideenfindungsphase
- **16:00 – 17:30:** Projektphase

1. Einführung

Was ist KI?

Moderne Anwendungen



Supervised vs Unsupervised

Überwachtes Lernen:

Ausgangspunkt: Gelabelte Daten

Ziel: Vorhersage des Labels y mithilfe der Eingabedaten X

Mögliche Daten:

X :



y : Apfel

X :

Luftdruck: 1,2 bar
Luftfeuchtigkeit: 70%
Wind: 10km/h
Regen: Nein

y : 23°C

Unüberwachtes Lernen:

Ausgangspunkt: Daten

Ziel: Lernen von Mustern innerhalb der Eingabedaten X

Mögliche Daten:

Kunde A: Eier, Brot, Milch
Kunde B: Eier, Brot, Wasser
Kunde C: Wasser, Ofen, Pizza

→ Clustering von Kunden

Supervised Learning

Regression:

Anhand von Merkmalen einen kontinuierlichen Wert vorhersagen

Beispiel:

Merkmale: Luftdruck, Feuchtigkeit, Wind
Vorhersage: Temperatur

Merkmale: Lage, Größe, Anzahl Zimmer
Vorhersage: Mietpreis

Klassifikation:

Anhand von Merkmalen den Datenpunkt einer Klasse zuordnen.

Beispiel:

Merkmale: Kontostand, Einkommen
Vorhersage: Kreditwürdig? Ja oder nein

Merkmale: Größe, Farbe, Form
Vorhersage: Welches Obst?

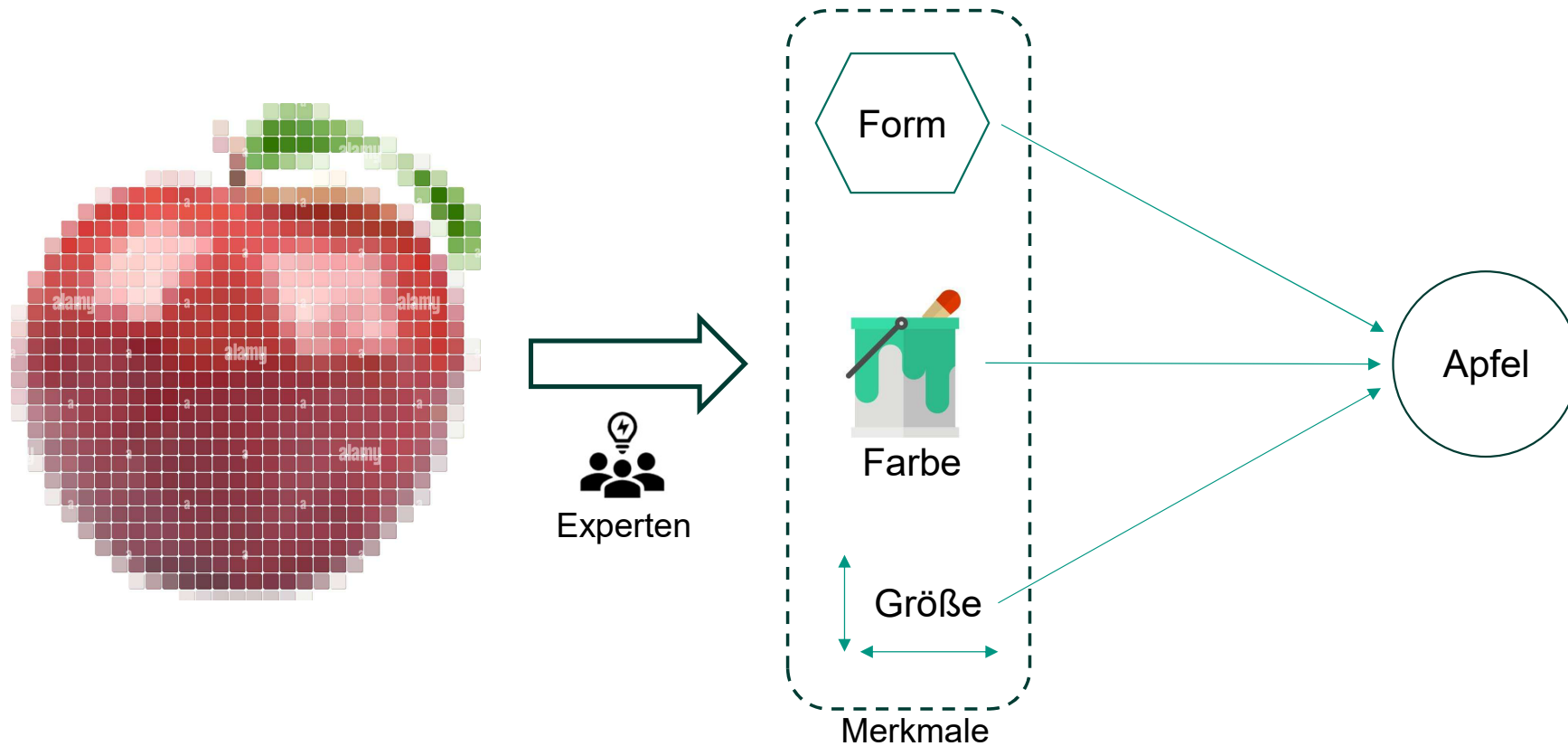
2. Neuronale Netze

2.1. Aufbau

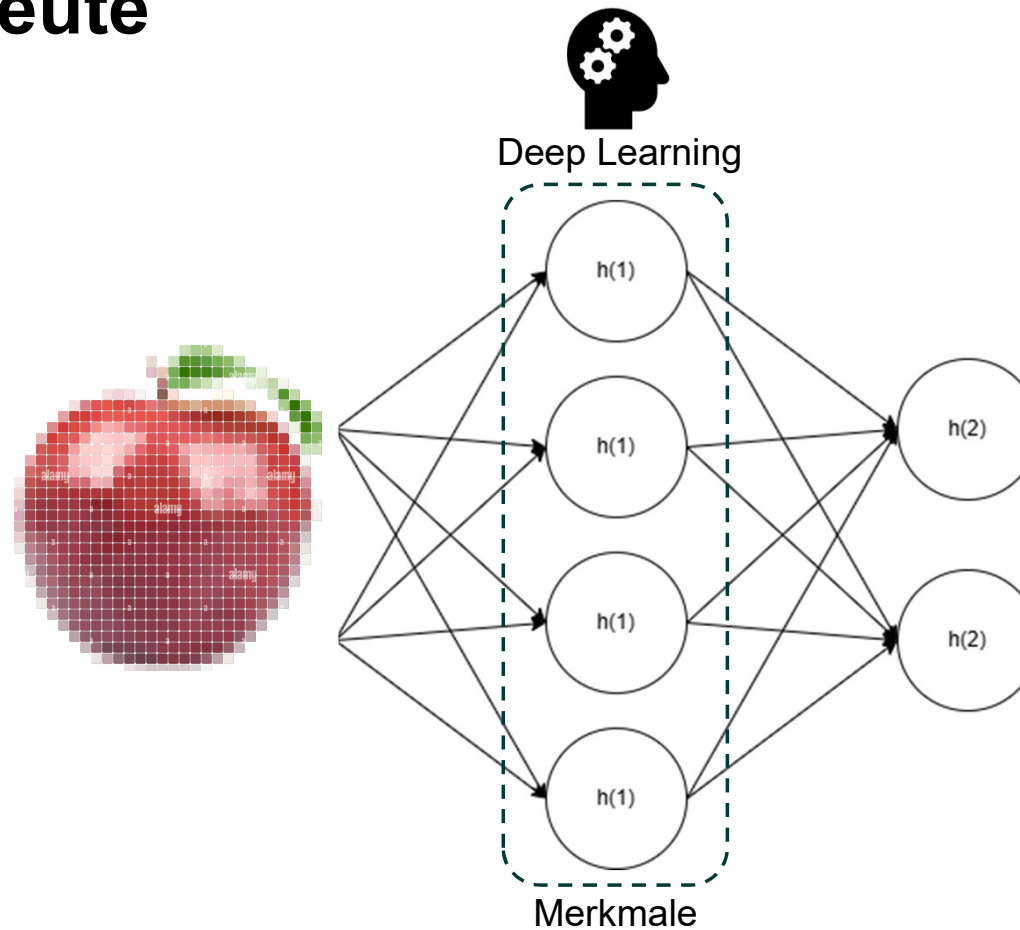
2.3. Aktivierungsfunktionen

2.2. Training

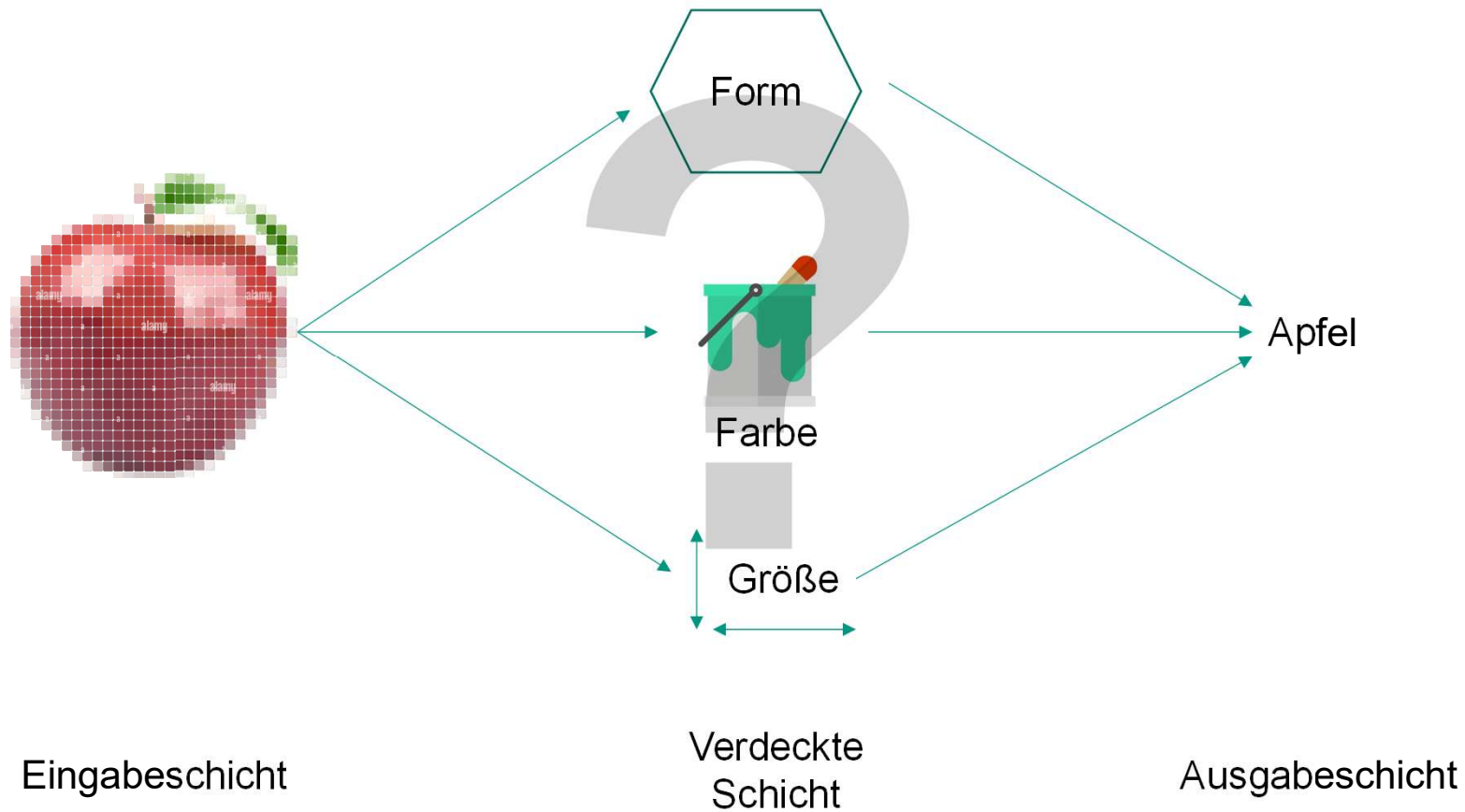
Lernen - Früher



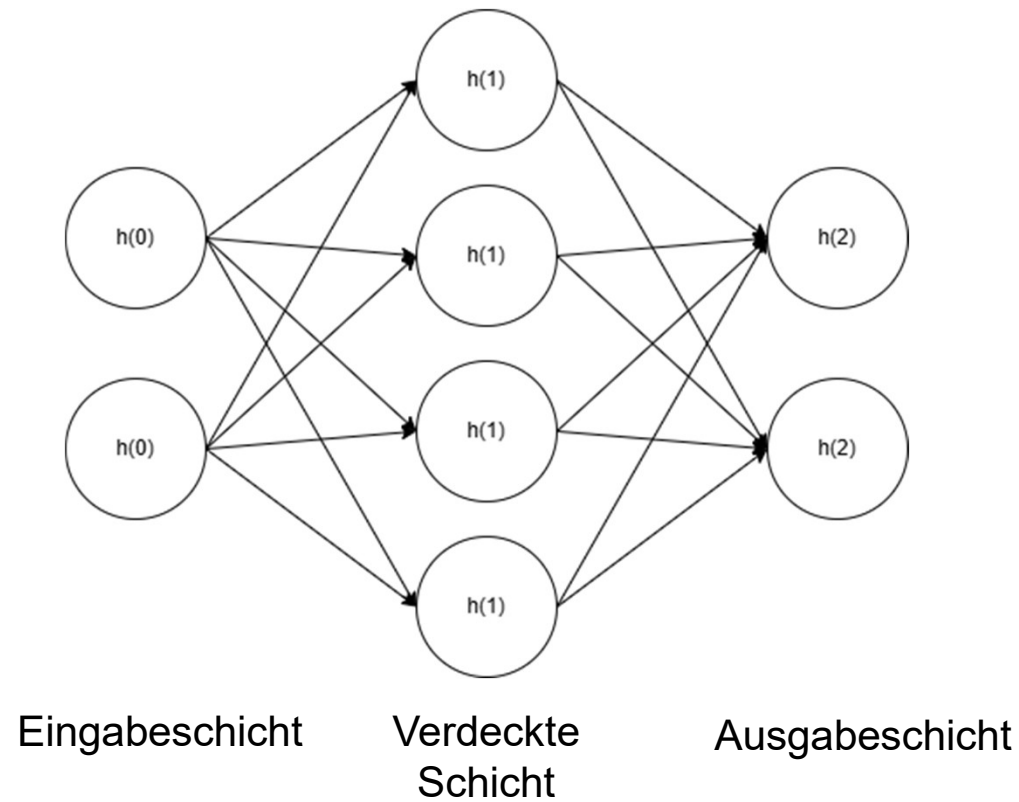
Lernen - Heute



2.1 Aufbau

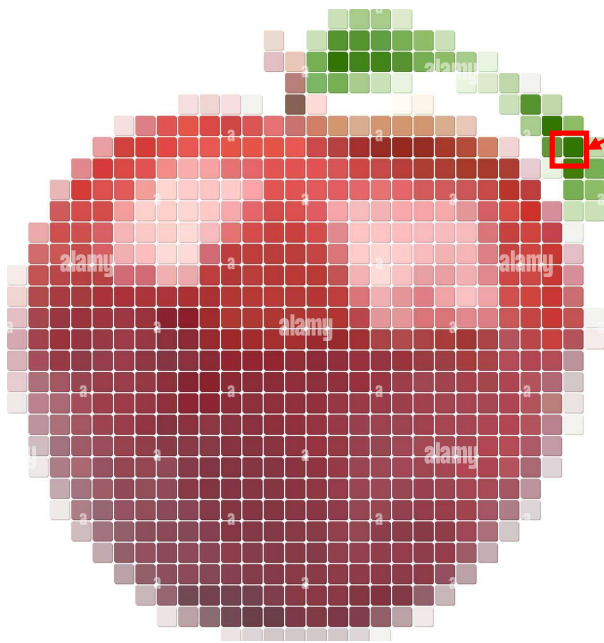


2.1 Aufbau



2.1 Aufbau - Bilderkennung

<https://www.alamy.de/verpixelter-roter-apfel-image463022146.html>



36 x 28 Pixel

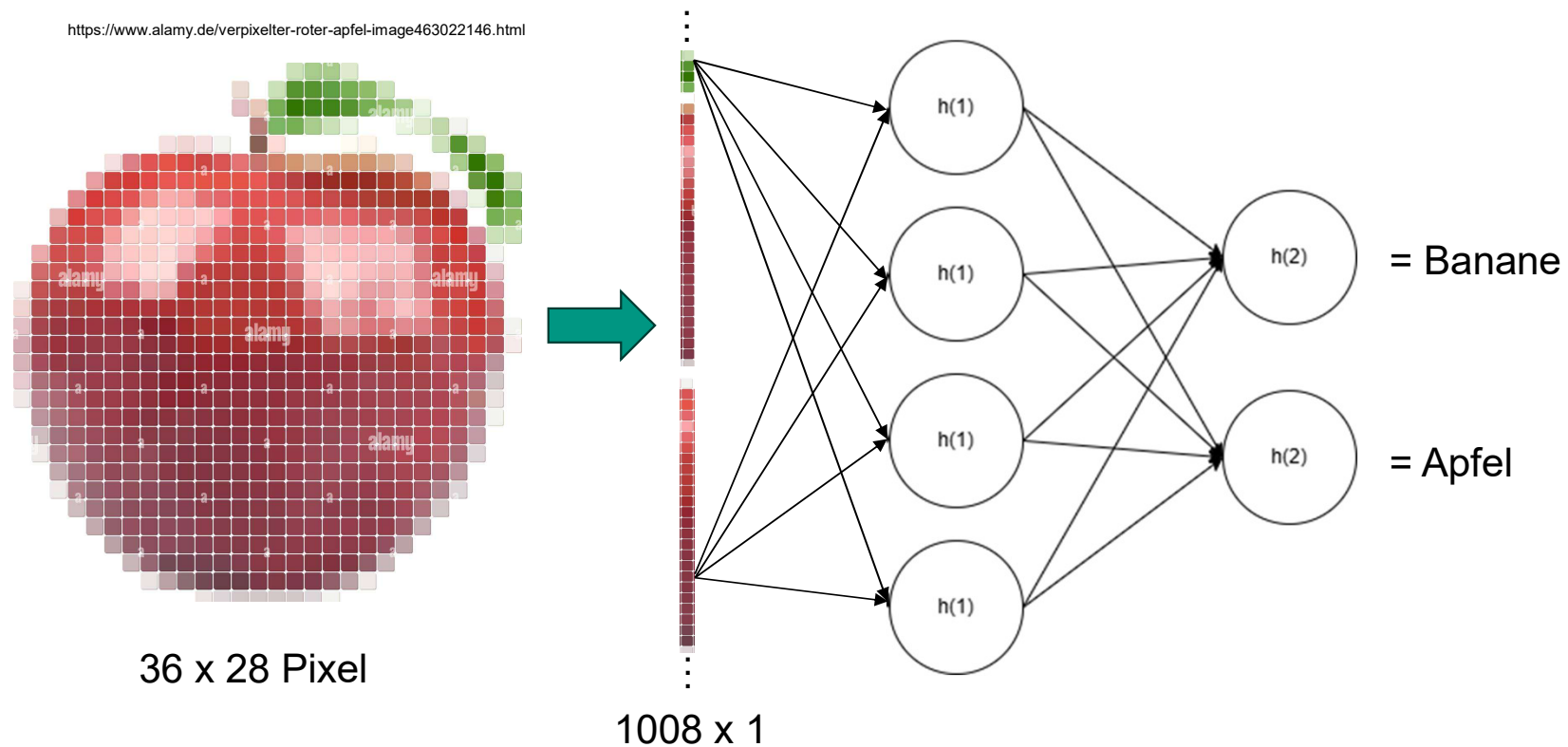
Jedem Pixel kann durch Faktoren wie Helligkeit/Intensität und Farbe ein numerischer Wert zugeordnet werden

Der Wert des Pixels dient als numerischer Eingabewert eines Input-Neurons

Wie viele Neuronen benötigt das Netz in der Eingabeschicht um dieses Bild zu klassifizieren?

→ 1008

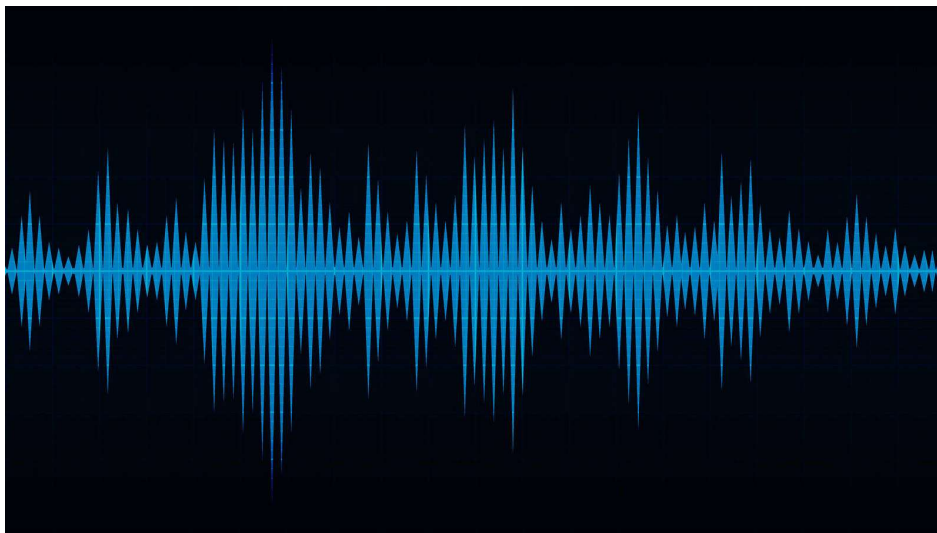
2.1 Aufbau - Bilderkennung



2.1 Aufbau - Spracherkennung

Wie könnte Spracherkennung funktionieren?

<https://autozubehor-gdl0.webnode.mx/audio/>



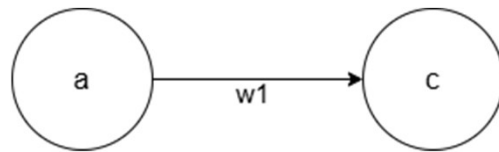
MFCC Spectrogram for "Hello world"

→ Bilderkennung!

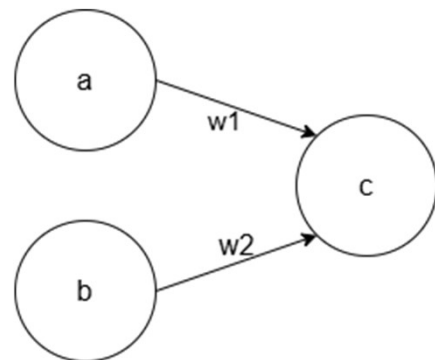
2.2 Lernen

Wie kann eine Verkettung von Neuronen nun etwas lernen?

2.2 Lernen



$$C = w1 * a$$



$$C = w1 * a + w2 * b$$

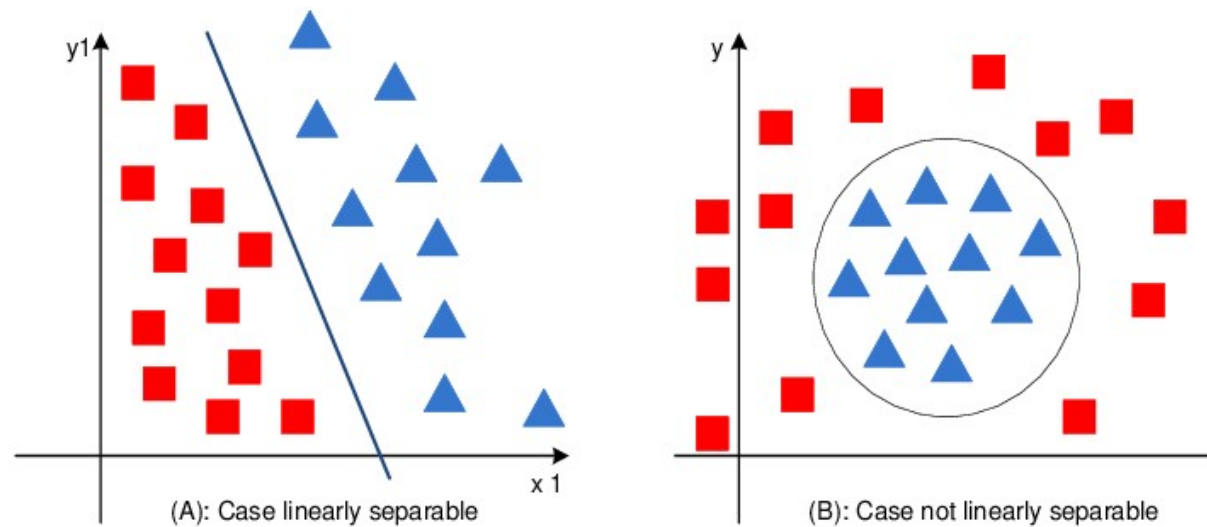
2.2 Lernen

Wieso sollte ich eine nichtlineare Aktivierungsfunktion nutzen?

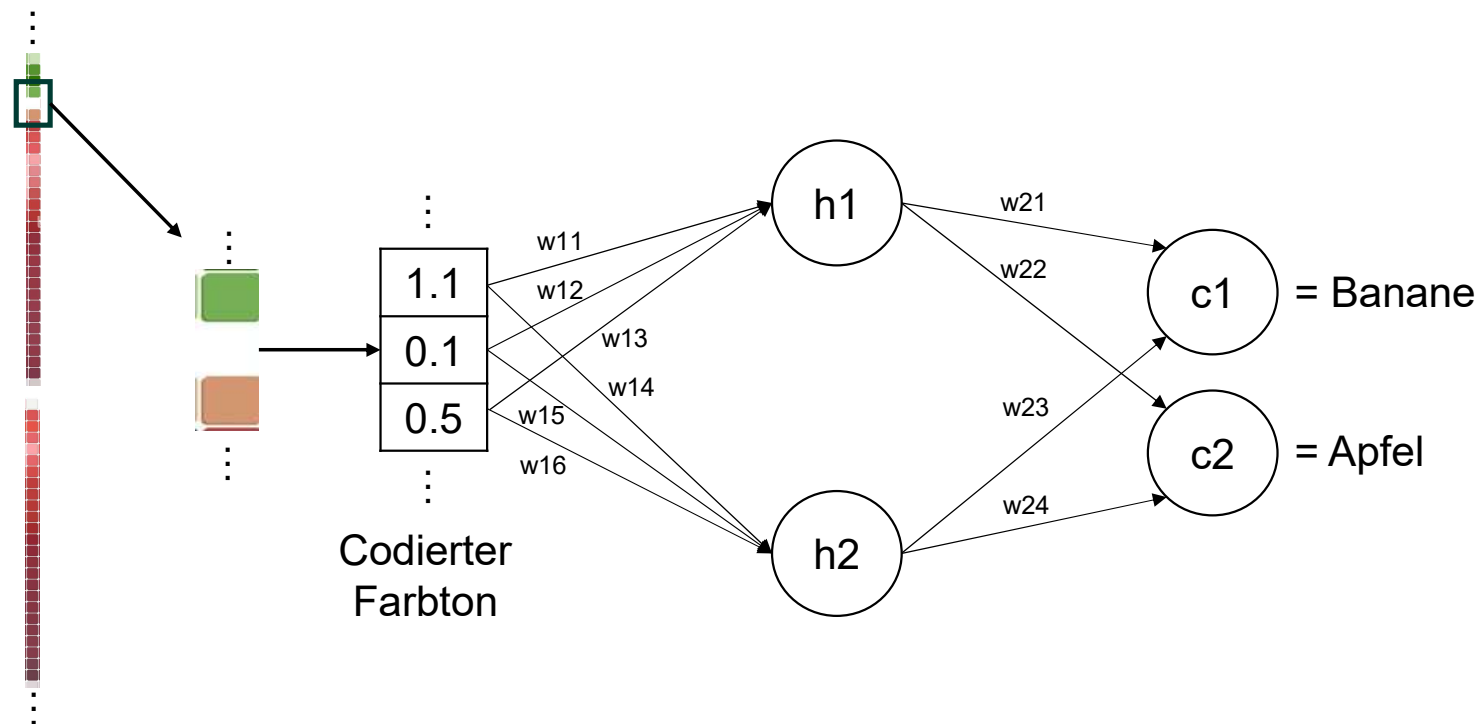


2.2 Lernen

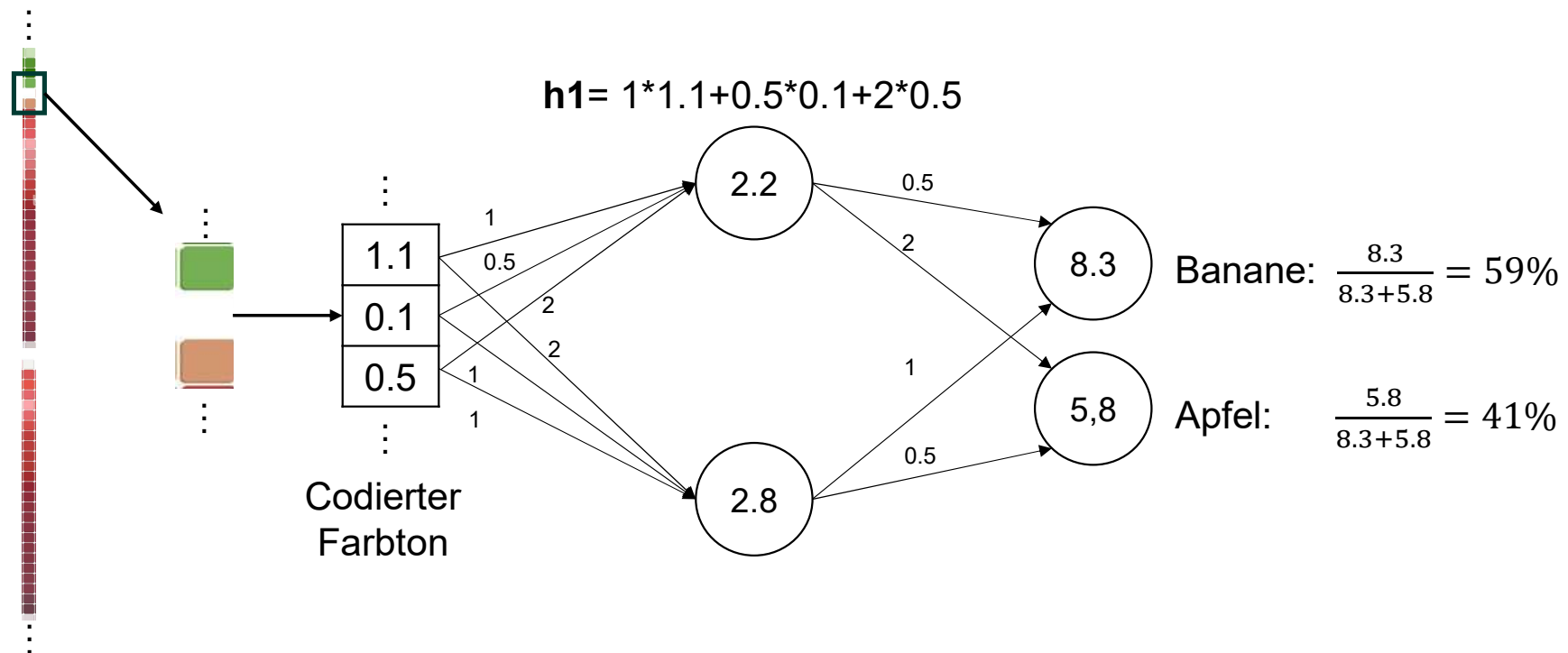
Wieso sollte ich eine Aktivierungsfunktion nutzen?



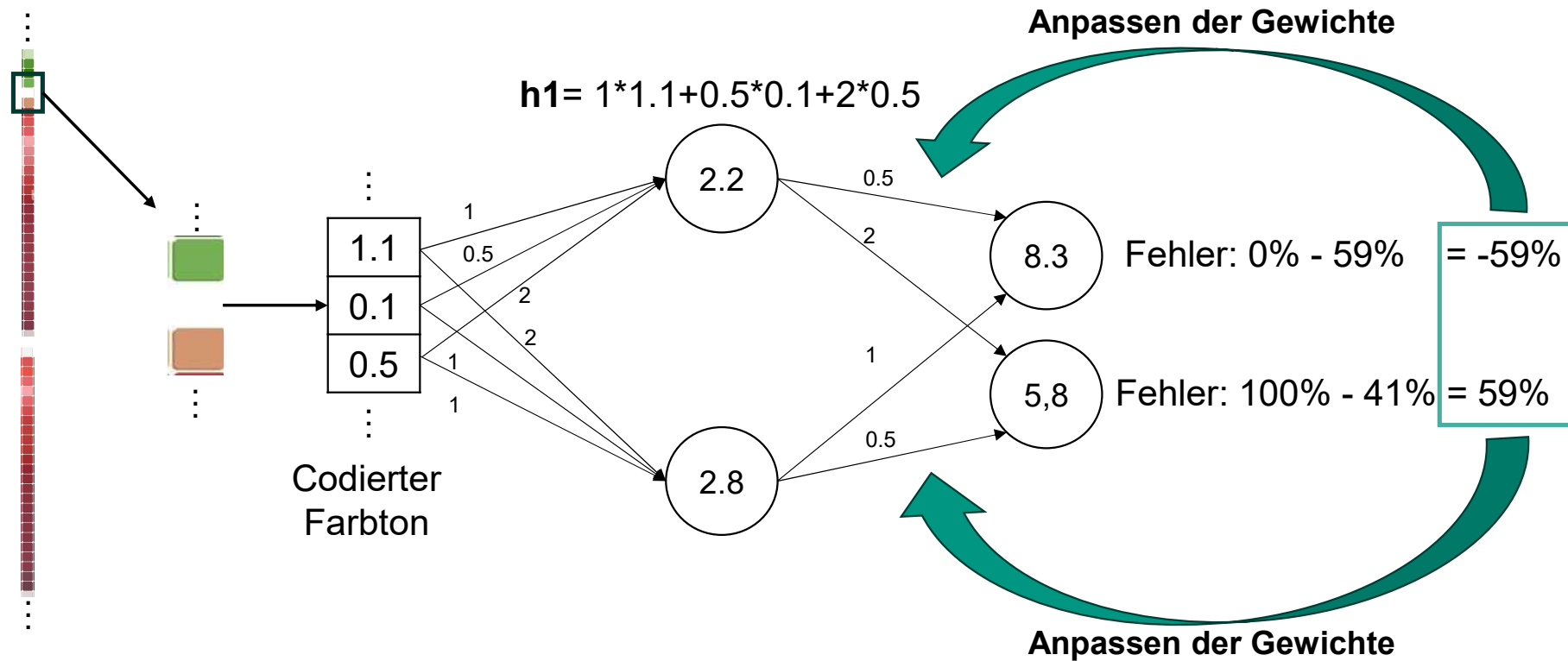
2.2 Lernen



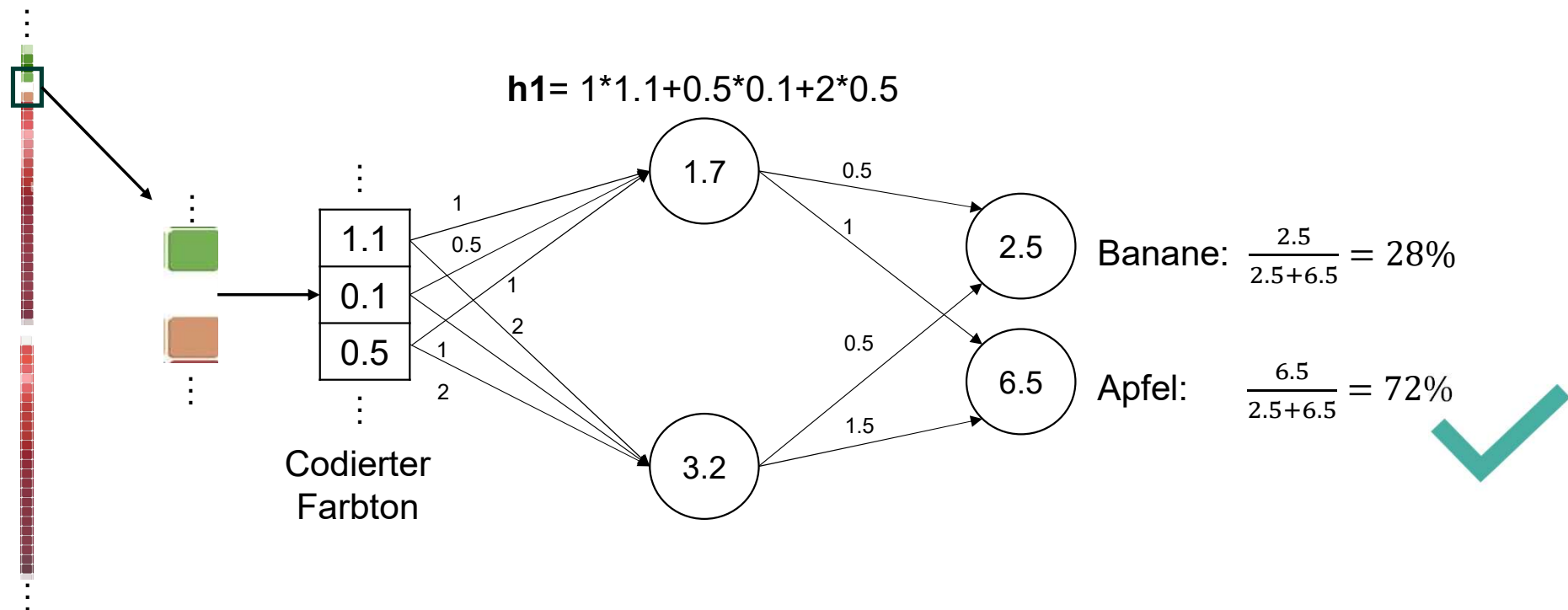
2.2 Lernen



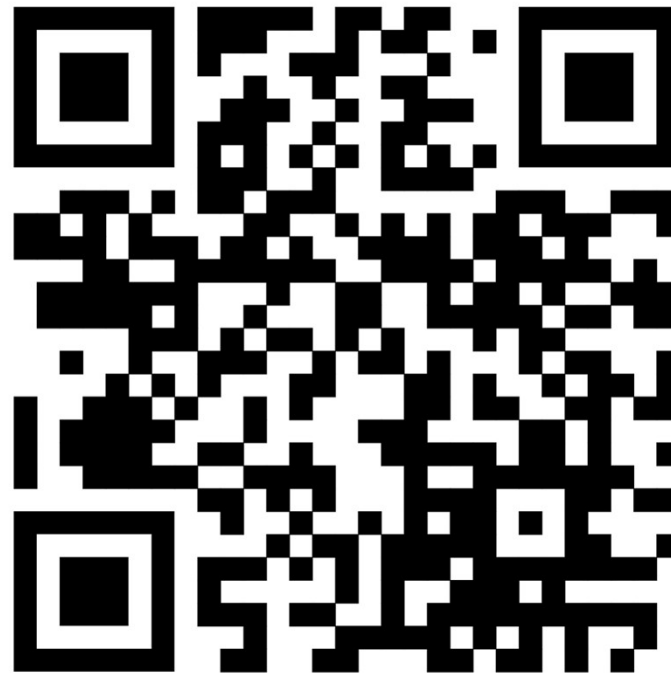
2.2 Lernen



2.2 Lernen



Arbeitsphase 1



<https://github.com/Tarn017/AI-Lab>

3.1 Wichtige Parameter

Neben den Parametern die gelernt werden gibt es noch weitere Hyperparameter die selbst gewählt werden müssen

3.1 Wichtige Parameter

1. Datenmenge:

Für das Training reicht ein einziger Datenpunkt/Bild nicht aus. Eine ausreichende Datenmenge ist für ein stabiles Modell unverzichtbar

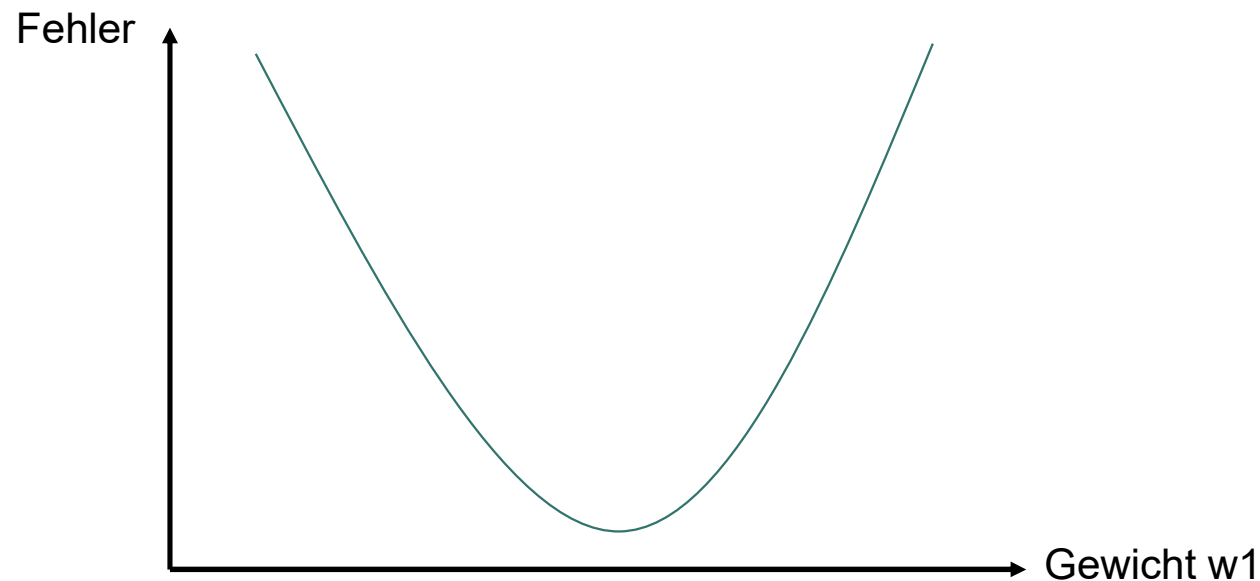
2. Epochen:

Der Fehler über alle Daten kann normalerweise nicht in einem einzigen Durchlauf minimiert werden. Es benötigt mehrere Durchläufe bzw. Epochen, in denen der Fehler immer weiter reduziert wird.

3.1 Wichtige Parameter

3. Lernrate:

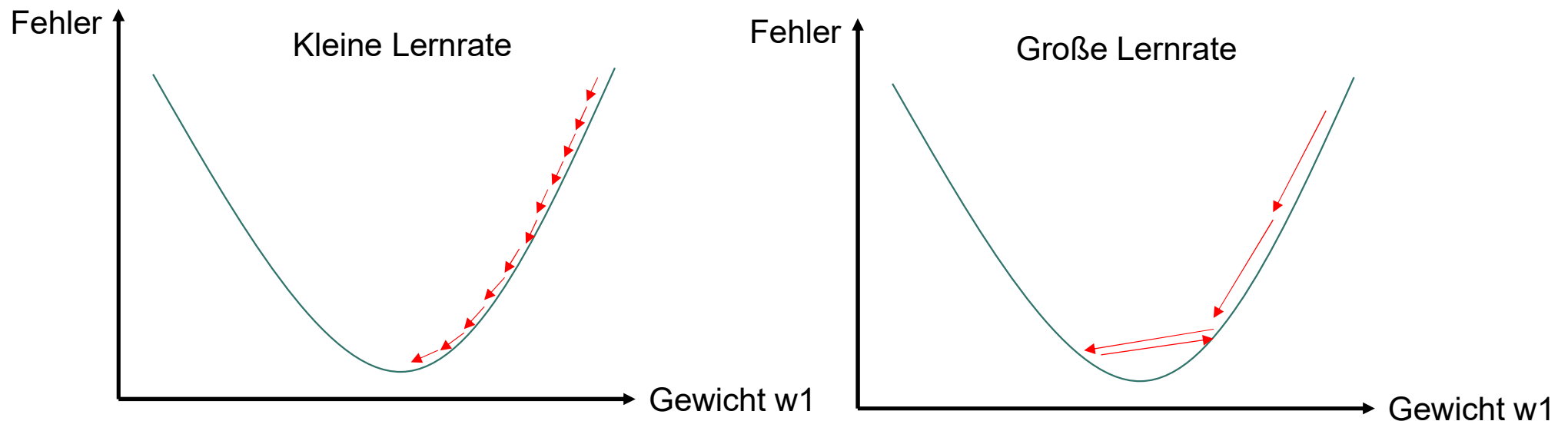
Die Lernrate gibt an wie stark die Gewichte basierend auf dem Fehler angepasst werden sollen. Sollten diese in großen, schnellen Schritten angepasst werden oder eher in kleinen, feinen Schritten?



3.1 Wichtige Parameter

3. Lernrate:

Die Lernrate gibt an wie stark die Gewichte basierend auf dem Fehler angepasst werden sollen. Sollten diese in großen, schnellen Schritten angepasst werden oder eher in kleinen, feinen Schritten?



3.1 Wichtige Parameter

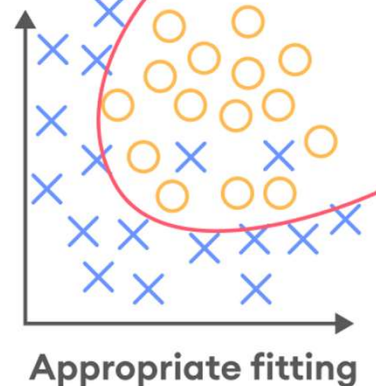
4. Anzahl & Größe der Schichten:

Je mehr Schichten im Netz und je größer die einzelnen Schichten, desto komplexere Abhängigkeiten können gelernt werden. Gleichzeitig steigt jedoch die Gefahr, dass statt die unterliegende Struktur gelernt, lediglich die gesehenen Daten auswendig gelernt werden. Das führt zu einer schlechten Leistung in der Realität.

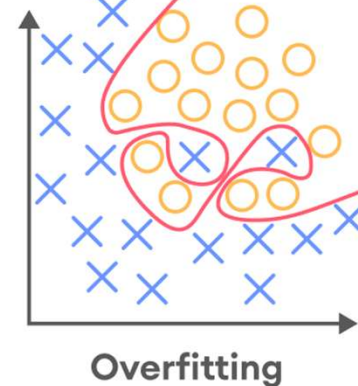
3.2 Overfitting & Regularisierung

Das Problem, dass Trainingsdaten nur auswendig gelernt werden wodurch das Modell später in der Realität schlecht abschneidet wird Overfitting genannt.

Tatsächliche Funktion
zur Trennung der
Klassen



Trainingsdaten werden
kompliziert auswendig
gelernt



[Overfitting and underfitting in machine learning | SuperAnnotate](#)

3.2 Overfitting & Regularisierung

Wie kann ich frühzeitig feststellen, ob mein Modell unter Overfitting leidet?

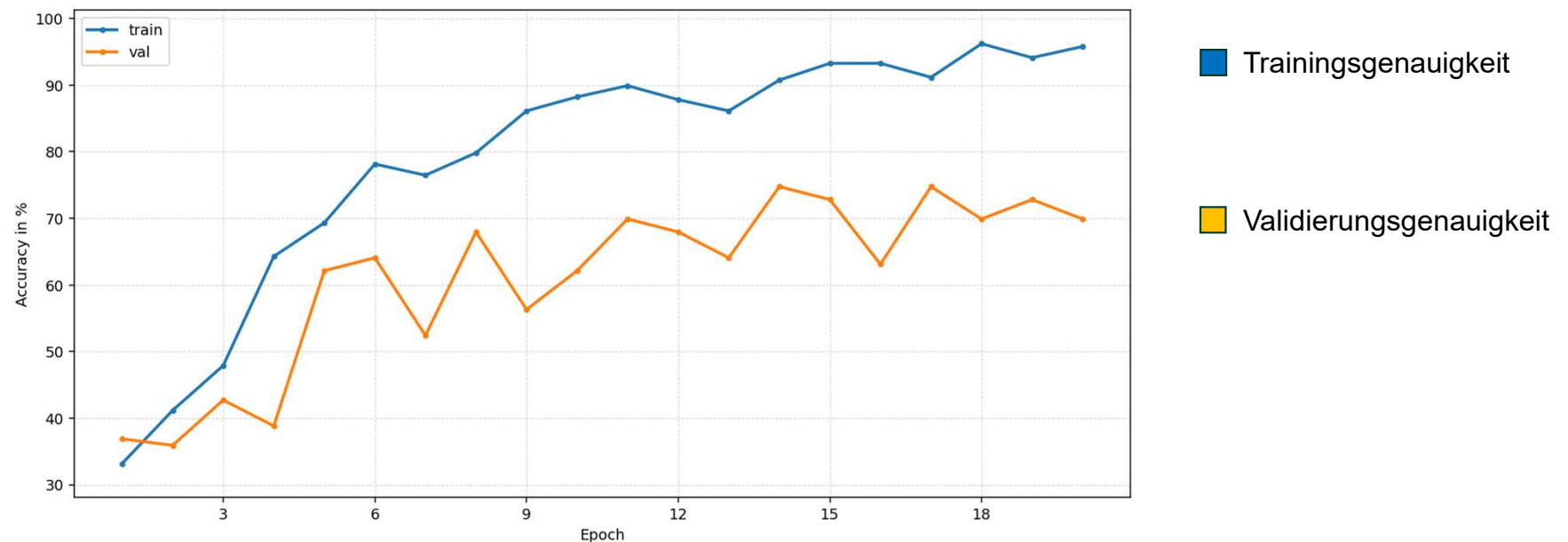
Teile den Datensatz in Trainings- und Validierungsdatsatz auf. Trainiere nun nur auf dem Trainingsdatensatz und berechne die Genauigkeit des Modells auf den Trainingsdaten. Berechne nun die Genauigkeit auf dem Validierungsdatsatz.

Trainingsgenauigkeit >> Validierungsgenauigkeit → Overfitting

3.2 Overfitting & Regularisierung

Wie kann ich frühzeitig feststellen, ob mein Modell unter Overfitting leidet?

Noch besser: Bereits während des Trainings die Validierungsgenauigkeit überwachen



3.2 Overfitting & Regularisierung

Wie kann ich Overfitting verhindern?

- Netz verkleinern
- Dropout
- Data Augmentation

3.2 Overfitting & Regularisierung

Dropout:

Was ist besser als ein einziges Netz zu trainieren?

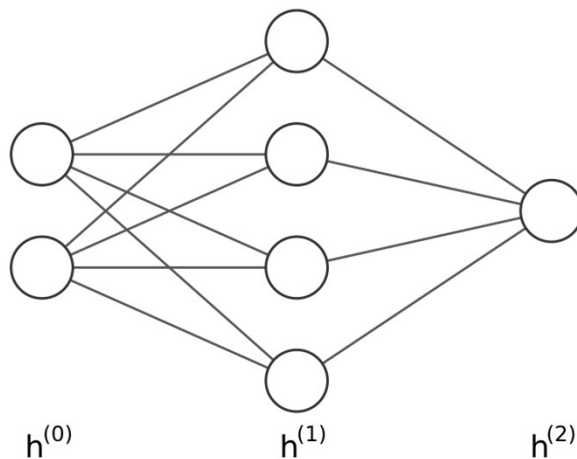
→ Mehrere Netze zu trainieren! Das Ergebnis wird robuster!

Tatsächlich mehrere Netze zu trainieren würde allerdings zu viel Rechenaufwand kosten. Darum erhält jedes Neuron im Netz nun eine Wahrscheinlichkeit während einer Epoche im Training deaktiviert zu werden.

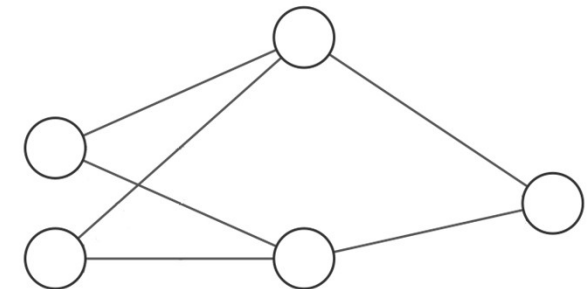
3.2 Overfitting & Regularisierung

Dropout:

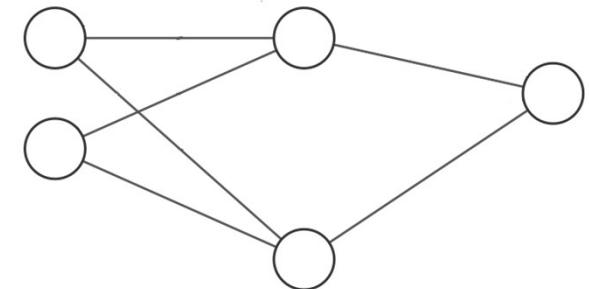
Ursprüngliches Netz:



Netz in Epoche 1:



Netz in Epoche 2:



Die Stärke der Regularisierung wird durch die **Dropoutrate p** gesteuert.

3.2 Overfitting & Regularisierung

Data Augmentation:

Transformieren eines Bildes, sodass eine höhere Variation innerhalb der Trainingsdaten entsteht. Mehr verschiedene Daten lassen das Modell robuster werden.



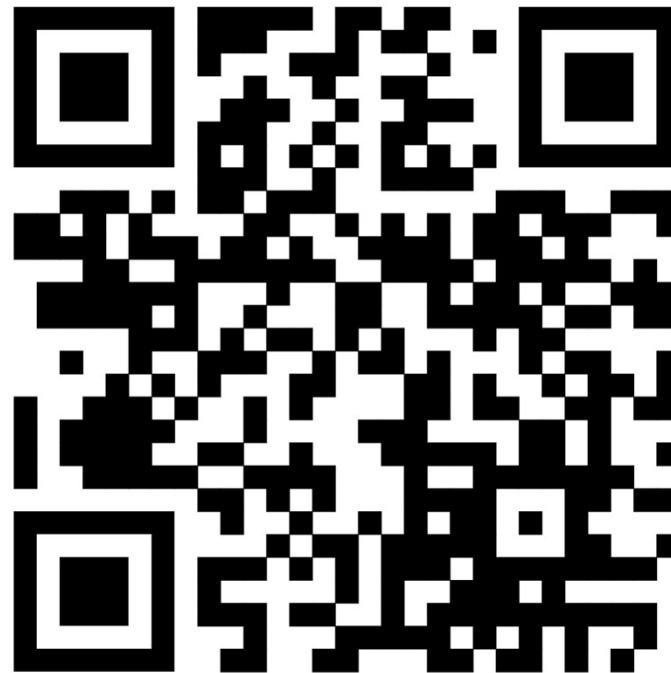
3.2 Overfitting & Regularisierung

Data Augmentation:

Achtung, in welchen Fällen müssen wir hier aufpassen?

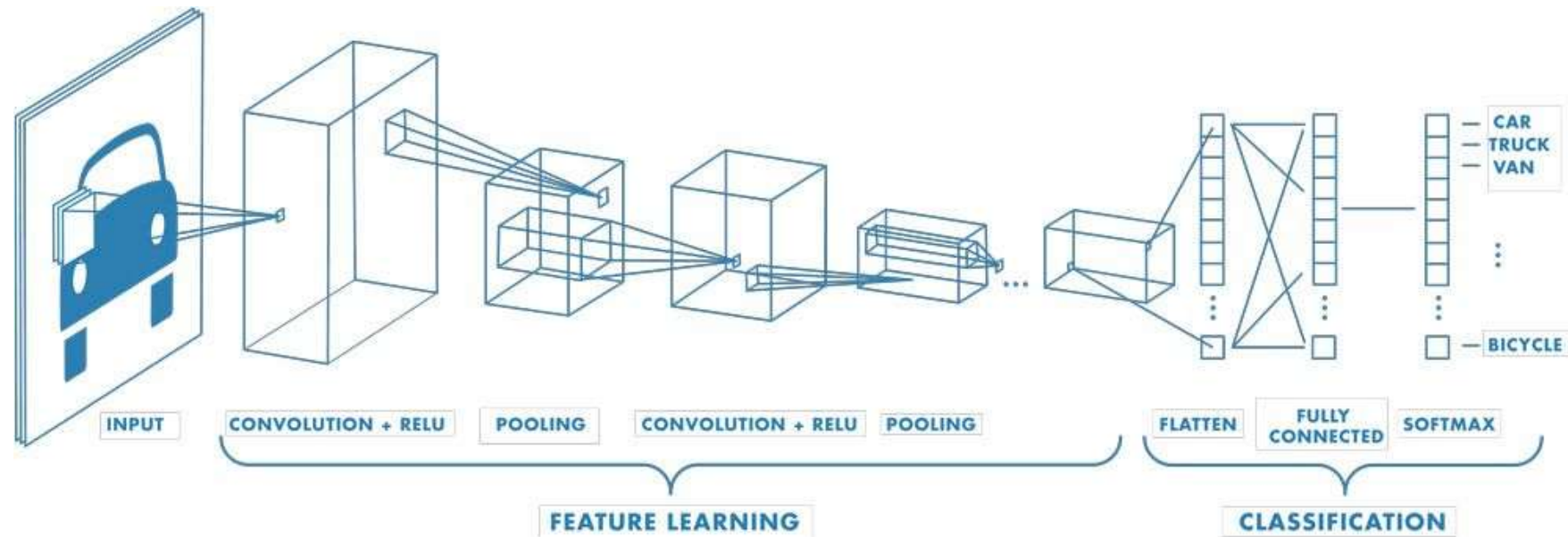


Arbeitsphase 2



<https://github.com/Tarn017/AI-Lab>

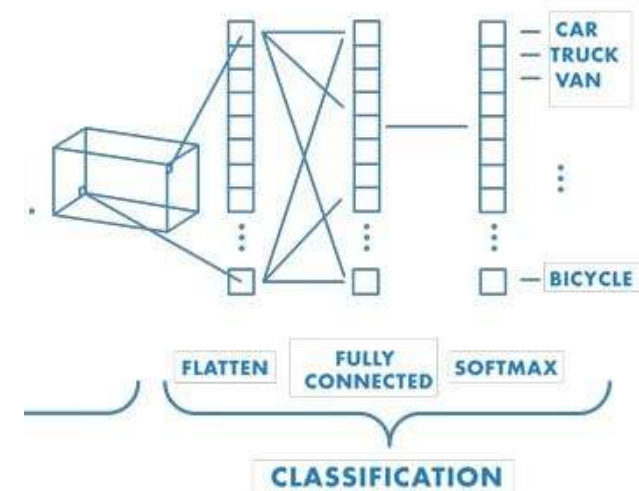
4. Convolutional Neural Network (CNN)



<https://de.mathworks.com/discovery/convolutional-neural-network.html>

4.1 CNN: Aufbau

Der hintere Teil entspricht dem klassischen NN
-> Nur der vordere Teil muss untersucht werden!



<https://de.mathworks.com/discovery/convolutional-neural-network.html>

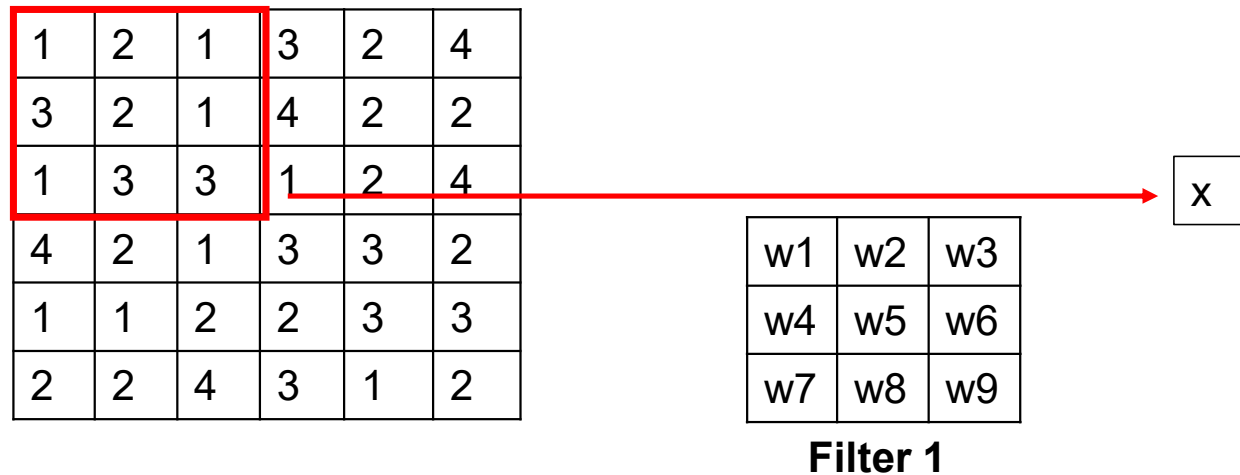
4.1 CNN: Aufbau



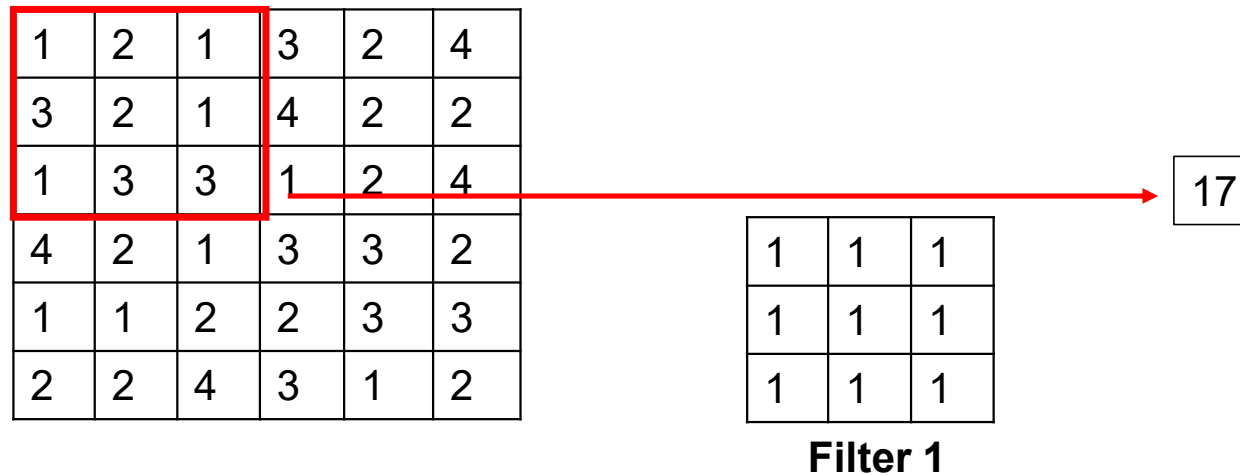
Eingabe ist ein Bild mit einer beliebigen Anzahl an Pixeln, bspw. 28x28. Jeder Pixel besitzt 3 Kanäle (Merkmale). Diese sind die Farbe der Pixels in RGB codiert (rot – grün –blau)

<https://de.mathworks.com/discovery/convolutional-neural-network.html>

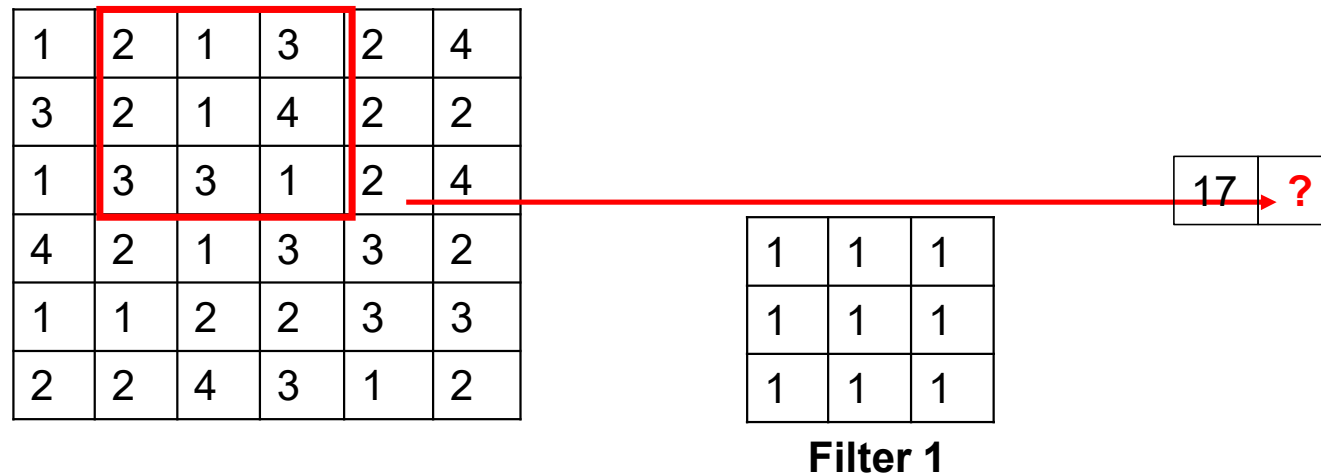
4.2 CNN: Convolutional Schicht



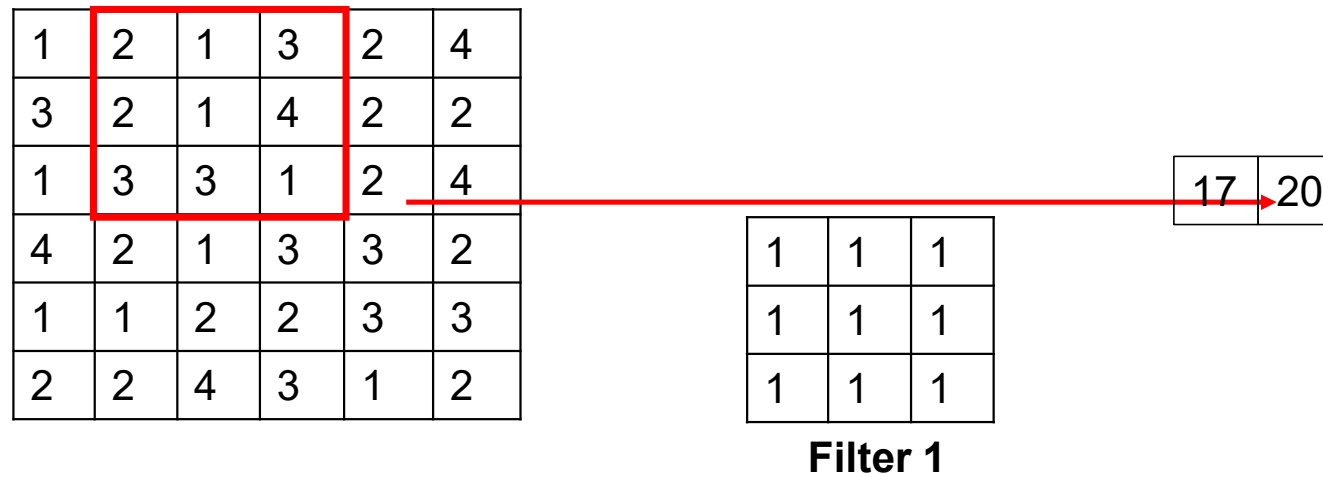
4.2 CNN: Convolutional Schicht



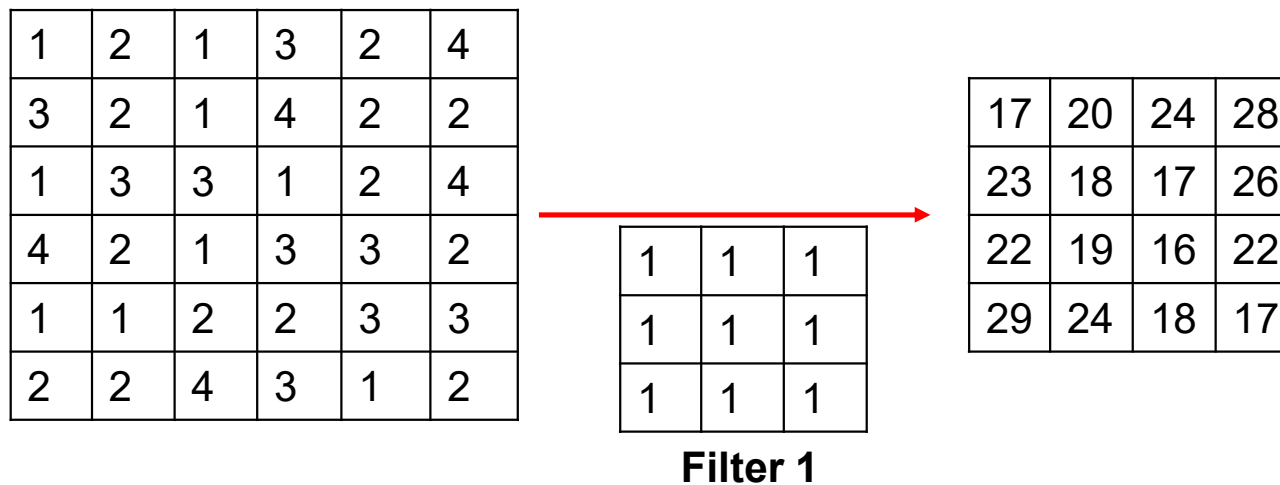
4.2 CNN: Convolutional Schicht



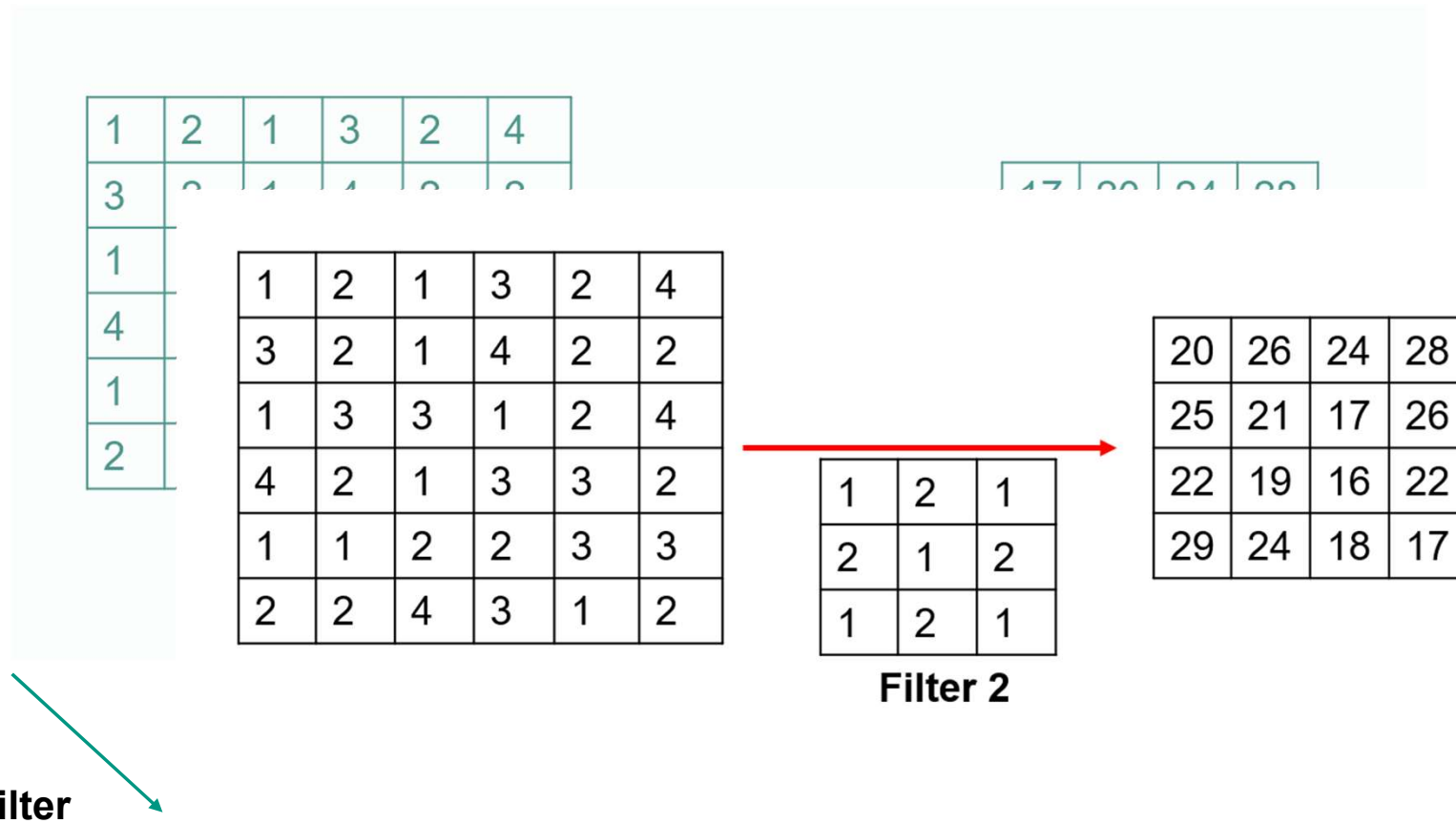
4.2 CNN: Convolutional Schicht



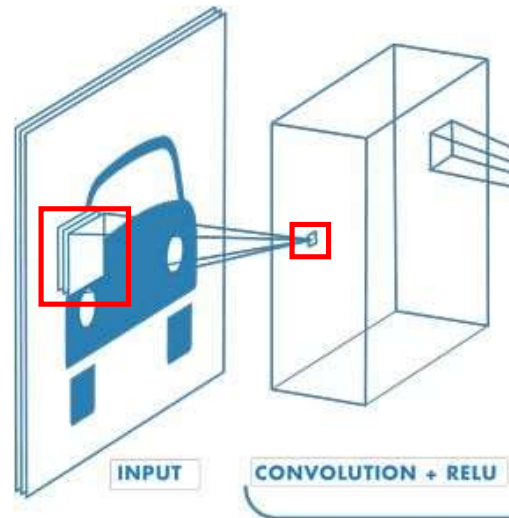
4.2 CNN: Convolutional Schicht



4.2 CNN: Convolutional Schicht



4.1 CNN: Aufbau

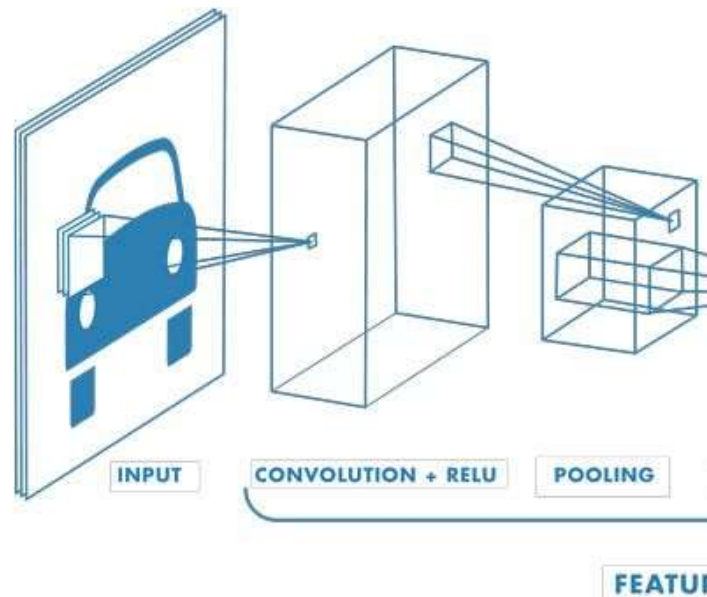


<https://de.mathworks.com/discovery/convolutional-neural-network.html>

Schicht 1: 6 Filter mit 3x3 Kernel

- Die Werte über 3x3 Pixel, sowie die 3 Eingabekanäle werden mit Gewichten multipliziert (hier insgesamt 27 Gewichte)
- Die 27 Werte werden addiert und damit auf einen einzigen Wert reduziert.
- Der Kernel wird weitergeschoben und erneut wird ein Wert berechnet, bis die nachfolgende Schicht vollständig ist.
- Wiederhole für die restlichen 5 Filter, sodass die nächste Schicht insgesamt 6 Kanäle dick ist.
- Jeder Filter lernt im Optimalfall ein anderes Merkmal.

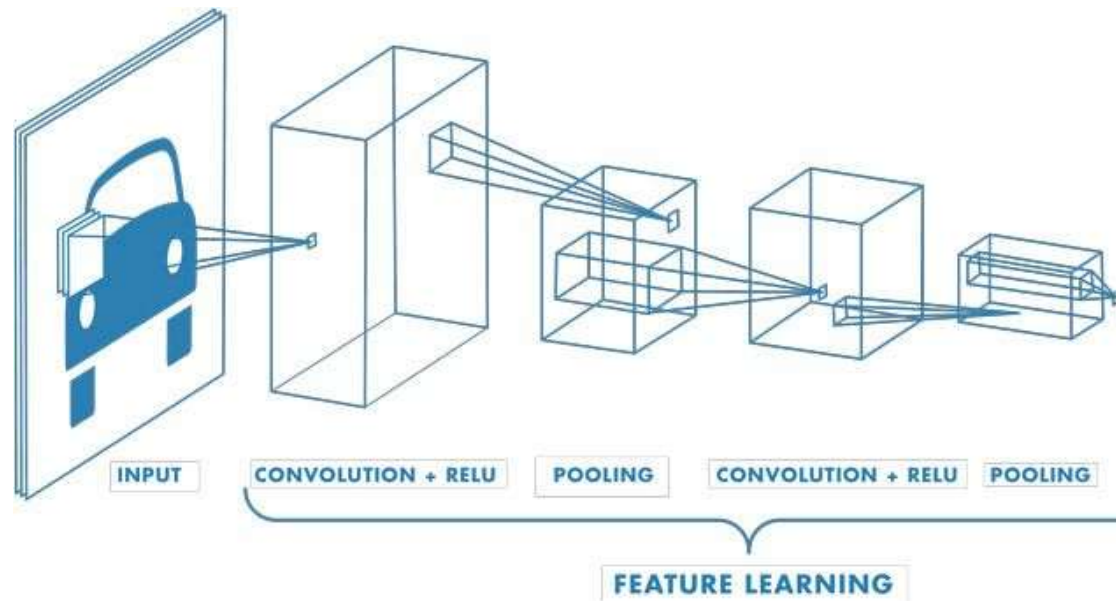
4.1 CNN: Aufbau



Die **Pooling Schicht** reduziert lediglich die räumliche Dimension des Bildes. Die Anzahl der Kanäle bleibt gleich. Bei einem Pooling Filter von 2x2 wird erneut ein Raster über das Bild gelegt. Die 4 Werte werden anschließend auf einen einzigen reduziert. MaxPooling: der höchste der 4 Werte wird übernommen

<https://de.mathworks.com/discovery/convolutional-neural-network.html>

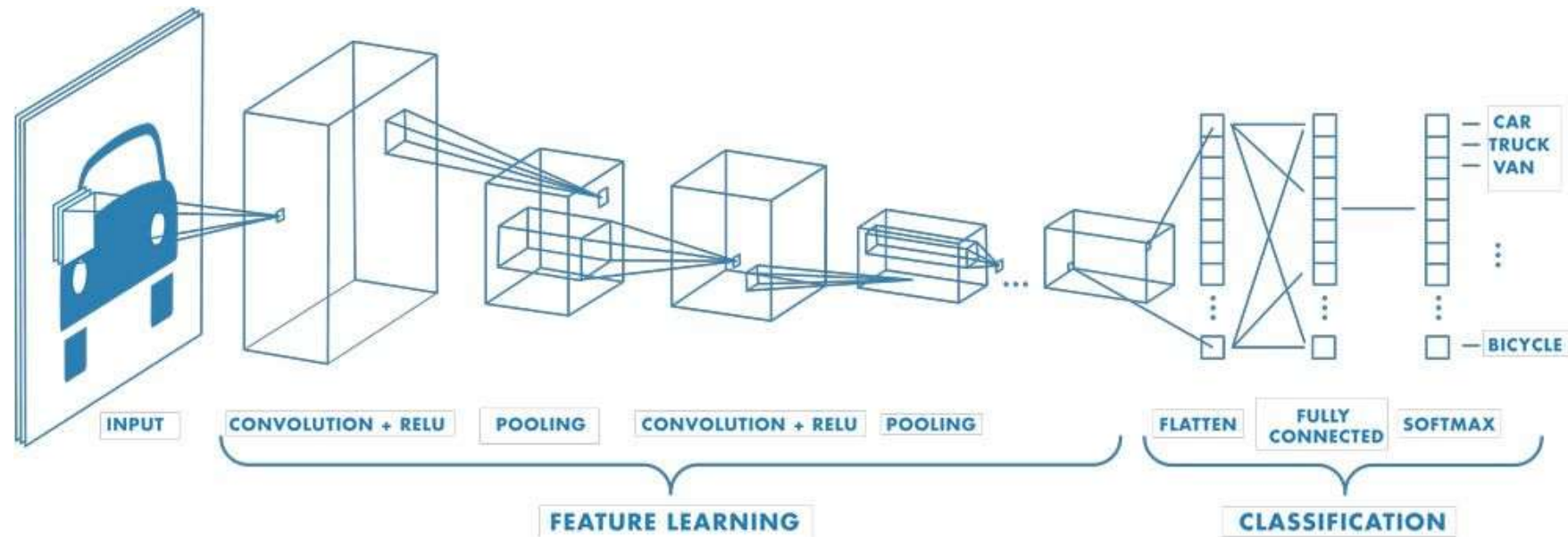
4.1 CNN: Aufbau



Wiederholt Convolutional- und Poolingschichten aneinanderreihen.

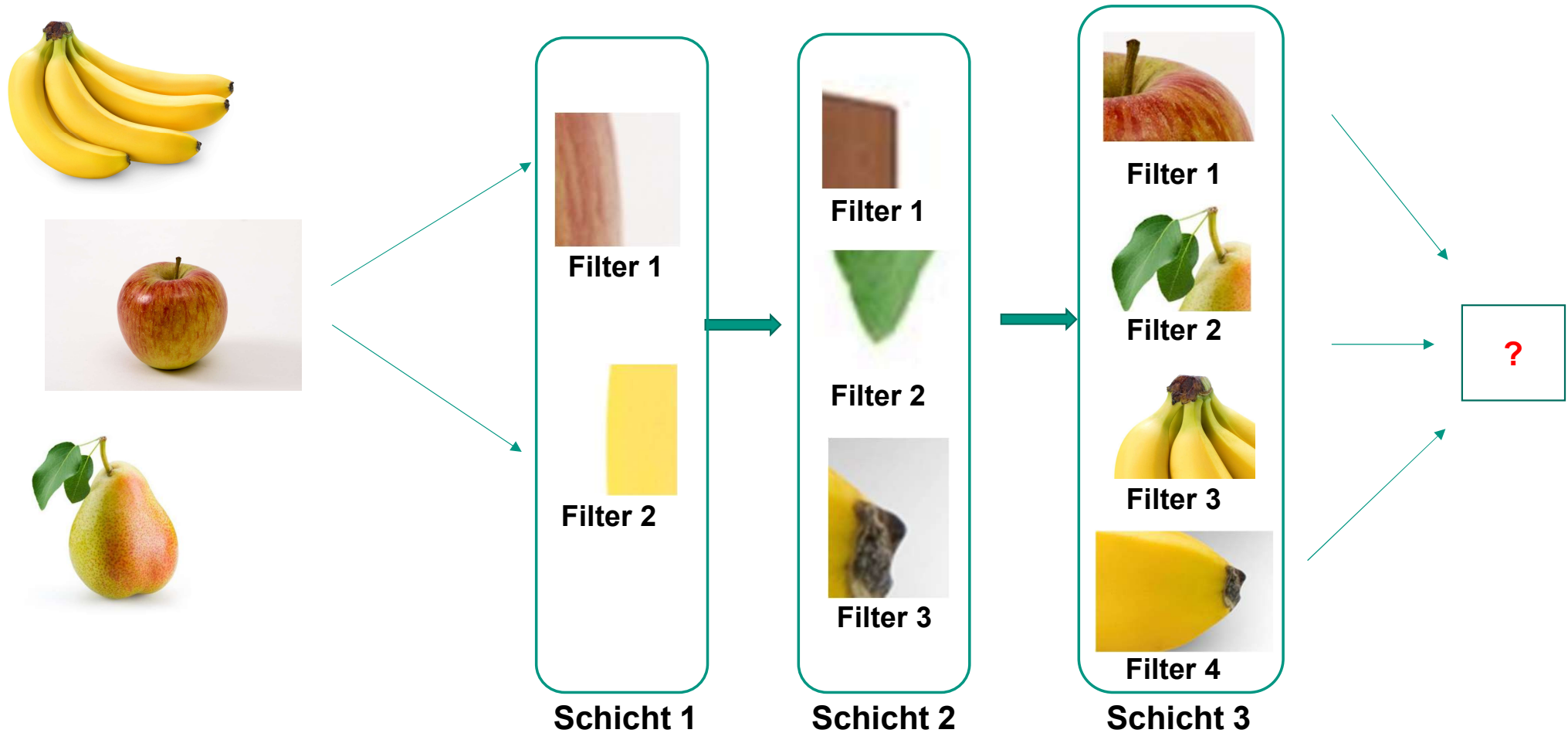
<https://de.mathworks.com/discovery/convolutional-neural-network.html>

4.1 CNN: Aufbau



<https://de.mathworks.com/discovery/convolutional-neural-network.html>

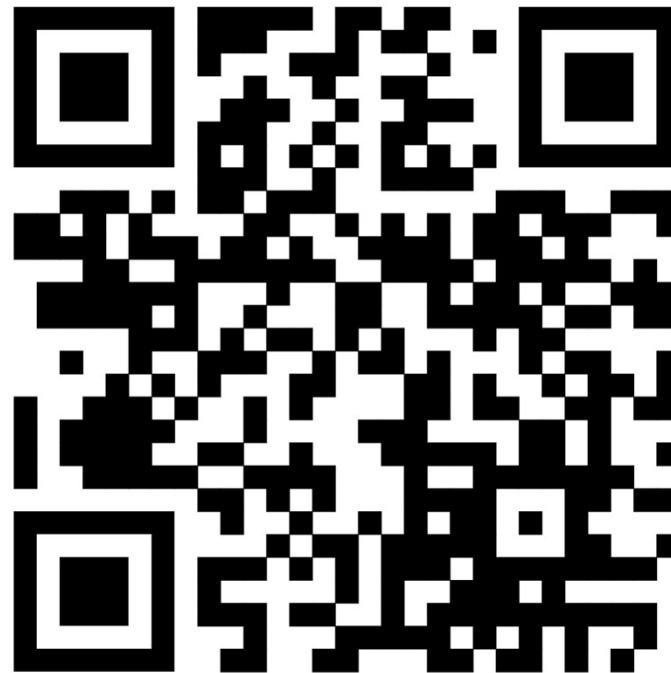
4.3 CNN: Lernen von Merkmalen



4.4 CNN: Zusammenfassung

- Wie auch im klassischen NN findet der Lernprozess über die Anpassung der Gewichte statt
- Nur ist ein Neuron in einer späteren Schicht nicht mehr mit allen Neuronen der vorherigen Schicht verbunden, sondern nur mit denen aus dem entsprechenden Filter
- Dieselben Filter werden für unterschiedliche Regionen eines Bildes genutzt weshalb universelle Merkmale gelernt werden

Arbeitsphase 3



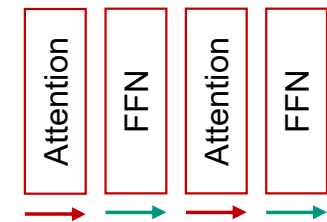
<https://github.com/Tarn017/AI-Lab>

Sind FFNs noch relevant?

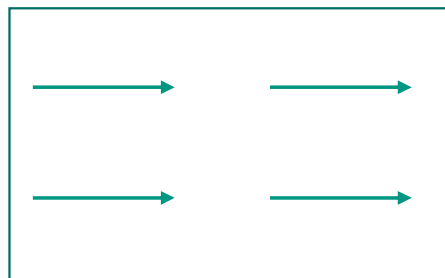
Feedforward
Netze (**FFN**)



Transformer:
Abwechselnd
Attention- und
FFN-Schichten



CNN: Aufbau
aus mehreren
kleinen FFNs



Moderne Strukturen

CNN zur Extraktion von Features, Transformer zur Verarbeitung:

